

Отчет по заданию «А5. Линейный алгоритм поиска всех простых чисел на отрезке $[1, N]$ »

Выполнил: студент Ошаров Александр группы БПИ248

Семинарист: Маров Руслан

Аннотация

В отчёте представлен линейный алгоритм (линейное решето Эратосфена) для нахождения всех простых чисел на отрезке $[1, N]$. Алгоритм основан на идее, что каждое составное число помечается ровно один раз — через его наименьший простой делитель. Приведён псевдокод и эквивалентная реализация на C++. Доказана корректность алгоритма и получено точное выражение временной сложности $T(N)$. С использованием определения асимптотической верхней границы показано, что $T(N) = O(N)$. В заключении отмечено, что дополнительно разработана и реализована более сложная версия на основе сегментированного решета с колесом, которая обладает лучшей константой, но имеет сложность $O(N \log \log N)$.

1. Постановка задачи

Дано целое положительное число N . Требуется:

1. Разработать линейный по времени и памяти алгоритм поиска всех простых чисел на отрезке $[1, N]$.
2. Представить описание алгоритма (в виде C++ кода).
3. Кратко обосновать корректность, используя свойство наименьшего простого делителя.
4. Составить точное выражение функции временной сложности $T(N)$.
5. Доказать, что $T(N) = O(N)$ по определению асимптотической верхней границы.

2. Линейный алгоритм (линейное решето)

2.1. Идея алгоритма

Ключевое наблюдение:

Каждое составное число x имеет единственный наименьший простой делитель p . Если мы будем помечать x только при обработке p , то каждое составное число будет обработано ровно один раз.

Алгоритм хранит:

- `is_prime[1..N]` — булев массив (или `vector<bool>`).
- `primes` — список найденных простых чисел.

Для каждого i от 2 до N :

- Если i не помечено как составное $\rightarrow$ добавляем в `primes`.
- Затем для каждого простого p из `primes`, такого что $p \leq \text{min_prime_factor}(i)$ и $p * i \leq N$:
 - Помечаем $p * i$ как составное.
 - Если p делит $i \rightarrow$ останавливаемся (иначе p не будет наименьшим делителем у $p*i$).

2.2. Реализация на C++

```
#include <vector>
```

```
using namespace std;
```

```
vector<int> linear_sieve(int N) {
    if (N < 2) return {};
    vector<bool> is_prime(N + 1, true);
    vector<int> primes;
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i <= N; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
        }
        for (int p : primes) {
            if (1LL * p * i > N) break;
            is_prime[p * i] = false;
        }
    }
}
```

```

        if (i % p == 0) break; // p — наименьший простой делитель i
    }
}

return primes;
}

```

Примечание: Внутренний цикл можно оптимизировать, используя индексацию, но приведённая версия сохраняет ясность.

3. Обоснование корректности

Рассмотрим любое составное число $x \leq N$. Пусть p — его наименьший простой делитель, тогда $x = p * q$, где $q \geq p$.

Во время итерации внешнего цикла при $i = q$:

- Все простые $\leq p$ уже находятся в `primes`.
- При обработке p в внутреннем цикле:
 - $p * q = x \leq N \rightarrow$ помечаем x как составное.
 - Так как p — наименьший делитель x , то p также делит q или $p <$ все делители q .
 - В любом случае, условие $q \% p == 0$ выполнится не позже, чем мы дойдём до p .
 - Следовательно, x будет помечено ровно один раз — при $i = q$ и простым p .

Таким образом:

- Все составные числа помечаются.
- Все простые числа не помечаются (они добавляются в `primes`).
- Каждое составное число обрабатывается один раз.

Алгоритм корректен.

4. Анализ временной сложности

4.1. Точное выражение $T(N)$

Ц Обозначим:

- Внешний цикл: $N - 1$ итераций ($i = 2..N$).
- Внутренний цикл: для каждого i выполняется столько раз, сколько простых p удовлетворяют:
 - $p \leq \text{min_prime_factor}(i)$
 - $p * i \leq N$

Но каждое составное число $x = p * i$ генерируется ровно один раз, и каждая такая генерация соответствует одной итерации внутреннего цикла.

Количество составных чисел на $[2, N] = N - 1 - \pi(N)$, где $\pi(N)$ — число простых $\leq N$.

Кроме того, для каждого простого i внутренний цикл выполняется хотя бы один раз (пометка $i * i$, если $i^2 \leq N$).

Однако ключевой факт:

Общее число итераций внутреннего цикла = количество составных чисел $\leq N$.

Почему? Потому что каждая итерация помечает одно уникальное составное число, и каждое составное число помечается один раз.

Следовательно:

$$T(N) = (N - 1) + (N - 1 - \pi(N)) = 2N - 2 - \pi(N)$$

(Первое слагаемое — внешний цикл, второе — внутренний.)

Но даже без точного подсчёта:

Количество итераций внутреннего цикла $\leq N$, так как каждая итерация соответствует уникальному $x = p * i \leq N$.

4.2. Доказательство $T(N) = O(N)$

По определению:

$T(N) = O(N)$, если существуют константы $C > 0$ и N_0 , такие что для всех $N \geq N_0$:
 $T(N) \leq C \cdot N$.

Из вышеизложенного:

- Внешний цикл: $\leq N$ итераций.
- Внутренний цикл: $\leq N$ итераций (по биекции с составными числами).
- Остальные операции — $O(1)$ на итерацию.

Следовательно:

$$T(N) \leq 2N \text{ для всех } N \geq 2.$$

Выбираем $C = 2$, $N_0 = 2$. Тогда для всех $N \geq N_0$:

$$T(N) \leq 2N = C \cdot N.$$

По определению, $T(N) = O(N)$.

5. Заключение

Разработан линейный алгоритм поиска всех простых чисел на отрезке $[1, N]$, известный как линейное решето Эратосфена. Алгоритм корректен благодаря использованию наименьшего простого делителя для уникальной пометки составных чисел. Временная сложность строго линейна: $T(N) = O(N)$, что доказано через подсчёт итераций и применение определения асимптотической верхней границы.

Примечание: В рамках самостоятельной работы дополнительно реализована более сложная версия алгоритма на основе сегментированного решета с колесом $W(30)$, оптимизированная для больших N и многопоточности. Эта реализация (файл `primes.cpp`) обладает меньшей константой и низким потреблением памяти, но имеет временную сложность $O(N \log \log N)$, а не $O(N)$. Она не удовлетворяет условию линейности, но может быть полезна на практике. Файл прилагается в том же архиве, что и настоящий отчёт.